CampusGig: A Verified-Student Micro-Freelancing Marketplace with Escrow

Project Proposal for UCS503P
Submitted to: Dr. Anushka
Thapar Institute of Engineering and Technology

Shashwat Narwal, CSED* Mohd Arham, CSED† Ineshvir Singh, CSED‡
Krish Gakhar, CSED§

August 19, 2026

1 Higher-order goal

. . . secondary framing

This project aims to make small-scale peer-to-peer work inside a campus economy safe and accountable, so that students can earn from verifiable skills without either party carrying the risk of non-payment or non-delivery. While the broader theme is *trust infrastructure for informal labour markets*, the immediate engineering objective is to translate an untracked cash-and-goodwill arrangement into a state-machine-driven software workflow with auditable money movement.

2 Time-to-value

. . . primary heuristic

- We will deliver meaningful increments quickly by prototyping the single core loop first (post → bid → accept → hold → deliver → release) and validating it with real student pairs within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations, keeping an automated regression suite green so that reworking the core loop stays cheap.
- Success is defined by what can be delivered and measured in the first iteration, then improved based on feedback from both sides of the marketplace.

3 Problem Statement

Students routinely need small paid work done (e.g., poster and resume design, video editing for club events, tutoring for a specific course, presentation formatting, basic technical support), and other students are willing and qualified to do it for a fee. Today this transaction happens through scattered hostel and batch WhatsApp groups, and the resulting market fails in predictable ways:

*Roll No: 1024240045, Email: snarwal_be24@thapar.edu
†Roll No: 1024240051, Email: marham_be24@thapar.edu
‡Roll No: 1024240067, Email: ikhurana_be24@thapar.edu
§Roll No: 1024240060, Email: kgakhar_be24@thapar.edu

- **No trust signal:** a poster seeking a video editor cannot distinguish a capable peer from an optimistic one, because there is no portfolio, completion history, or verified rating tied to a real identity.
- **No payment protection:** either the poster pays first and risks non-delivery, or the worker delivers first and risks non-payment; both sides absorb this risk informally.
- **No accountability:** if work is late, incomplete, or plagiarized, there is no record, no recourse, and no consequence carrying into the next transaction.
- **Discovery failure:** requests scroll away in group chats within hours, so demand and supply frequently fail to meet even when both exist on campus.
- **No dispute path:** disagreements are settled by argument or abandoned; the outcome depends on who is more persistent rather than on evidence.

Why existing solutions are insufficient: General freelancing platforms are built for a global market, so their fee structures, payout thresholds, and KYC/tax requirements are mismatched to a Rs. 200 poster design between two students in adjacent hostels. They also carry no campus-specific trust signal, which is precisely the primitive that matters here. Chat groups have reach but no state: they cannot represent an in-progress obligation, a held payment, or a completed transaction history.

Why this matters now (contextual relevance): The demand already exists and clears informally every day. The missing component is not marketplace liquidity but the trust and settlement layer, which is a tractable software problem.

4 Proposed Solution *. . . Software Proposition*

4.1 Overview

Build a **web-based** micro-freelancing marketplace restricted to verified students, with the following components:

- **Verified identity layer:** registration gated on institutional email, producing a profile carrying department, batch, and a non-transferable completion history.
- **Gig posting and discovery:** structured postings with category, scope, budget, and deadline, browsable and filterable rather than scrolling away.
- **Bidding and acceptance:** workers submit bids with a quoted price and delivery estimate; the poster accepts exactly one, which locks the terms.
- **Escrow ledger:** on acceptance, the agreed amount moves into a held state and is released only on poster confirmation, dispute resolution, or timeout.
- **Delivery and confirmation:** the worker submits deliverables through the platform; the poster confirms, requests revision, or raises a dispute within a bounded window.
- **Dispute and moderation console:** an administrator reviews the evidence trail and directs the held amount to one party or splits it, with the decision recorded.
- **Reputation:** ratings and completion statistics computed from settled transactions only, so reputation cannot be manufactured without completed gigs.

4.2 Core workflow

1. A verified poster creates a gig with category, scope, budget, and deadline.
2. Verified workers place bids; the poster reviews profiles, ratings, and history.
3. The poster accepts one bid; the agreed amount moves from available balance into escrow, and the gig enters an active state with frozen terms.
4. The worker submits deliverables before the deadline.
5. The poster confirms delivery (escrow releases to worker), requests a bounded number of revisions, or raises a dispute.
6. If no action is taken within the confirmation window, the system auto-releases to the worker to prevent indefinite fund capture.
7. On dispute, funds remain held until an administrator resolves the case; the outcome is logged against both profiles.
8. Both parties rate each other; ratings enter reputation only after settlement.

4.3 Operational constraints for an educational, campus setting

- **Escrow is implemented as an internal ledger, not a regulated payment service.** Holding third-party funds in India requires payment aggregator authorization, which is out of scope for a course project. The pilot therefore uses a closed-loop credit ledger with manual off-platform top-up and settlement, while the escrow state machine, invariants, and audit trail are built exactly as they would be against a real gateway.
- Keep the solution usable on low-to-mid range devices via responsive web design, since posting and bidding happen between classes rather than at a desk.
- Avoid dependence on advanced research algorithms; matching, ranking, and fraud checks are rule-based and explainable.
- Design for deployability using common web hosting, managed databases, and object storage for deliverables.

5 Solution Approach *... Engineering focus*

This is an engineering course project, so we focus on state correctness, transactional integrity, and access control rather than novel algorithm research.

5.1 Escrow as an explicit state machine *... correctness focus*

The central engineering artifact is a strictly enforced state machine over the contract that binds a gig to an accepted bid. The legal transitions are enumerated in Table 1; any transition not listed is rejected with HTTP 409 rather than silently ignored.

Two properties follow from this table and are enforced in code:

- **Funds are held exactly once.** Only the `OPEN` → `ACCEPTED` edge creates a hold, and it is guarded by a unique constraint permitting at most one non-terminal contract per gig.
- **Every exit from a holding state moves the full held amount.** The three terminal edges are the only ways to clear a hold, and each is checked to move exactly the held amount, so funds can neither be stranded nor duplicated.

From	To	Trigger	Ledger effect
OPEN	ACCEPTED	poster accepts bid	debit poster available, credit poster held
ACCEPTED	DELIVERED	worker uploads	none
DELIVERED	REVISION	poster requests	none
REVISION	DELIVERED	worker re-uploads	none
DELIVERED	RELEASED	poster confirms	debit poster held, credit worker available
DELIVERED	RELEASED	confirmation timeout	debit poster held, credit worker available
DELIVERED	DISPUTED	either party raises	none (funds stay held)
ACCEPTED	DISPUTED	deadline breached	none (funds stay held)
DISPUTED	RELEASED	admin rules	debit poster held, credit worker available
DISPUTED	REFUNDED	admin rules	debit poster held, credit poster available
DISPUTED	SPLIT	admin rules	debit poster held, credit both by stated shares

Table 1: Legal contract transitions and their ledger consequences. RELEASED, REFUNDED, and SPLIT are terminal.

5.2 Double-entry ledger design *... money representation*

Money is never stored as a mutable balance column. Instead:

- **Integer minor units.** All amounts are BIGINT paise, never floating point, so no rounding error can accumulate across splits.
- **Accounts.** Each user owns two accounts, AVAILABLE and HELD; the platform owns a single PLATFORM account used for fees (zero during the pilot) and for seeding pilot balances.
- **Entries.** A ledger entry is (txn_id, account_id, amount_paise, created_at), where debits are negative and credits positive. Rows are **append-only**: UPDATE and DELETE privileges on the table are revoked from the application role, so corrections are made by posting a compensating transaction, exactly as in accounting practice.
- **Balanced transactions.** Every txn_id must sum to zero. This is enforced by a DEFERRABLE INITIALLY DEFERRED constraint trigger that fires at commit, so a partially written transaction cannot be committed.
- **Derived balances.** A balance is $\sum$ of entries for an account. For read performance this is maintained as a materialized projection, but the ledger remains the single source of truth and the projection is verifiable against it at any time.

A release of Rs. 200 therefore writes exactly two rows under one txn_id: -20000 paise on the poster's HELD account and $+20000$ paise on the worker's AVAILABLE account.

5.3 Concurrency control and idempotency *... the hard part*

The failure mode that matters is two requests trying to settle the same held amount: for example, the poster tapping "confirm" while the timeout job auto-releases, or a double-tap on release. Our defences are layered:

- **Row-level locking.** Every state transition begins with SELECT ... FROM contracts WHERE id = $1 FOR UPDATE, so concurrent transitions on the same contract serialize. The

state is re-read *after* acquiring the lock, never before, so a losing request sees the already-updated state and is rejected by the transition table.

- **Atomic units of work.** The state update and its ledger entries are issued in one database transaction; there is no application-level "write state, then write money" window in which a crash could leave the two disagreeing.
- **Idempotency keys.** Money-affecting endpoints require an `Idempotency-Key` header. The key is stored with a unique index alongside the response; a replayed key returns the stored response instead of re-executing, so a retry on a flaky campus network cannot double-release.
- **Database-level guards.** A partial unique index (`WHERE state NOT IN terminal states`) permits at most one live contract per gig; `CHECK (amount_paise > 0)` rejects zero and negative amounts; foreign keys prevent orphan bids, entries, and deliverables.
- **Idempotent background jobs.** The auto-release sweeper selects only contracts in `DELIVERED` past their confirmation deadline and takes the same row lock, so it competes fairly with user requests rather than bypassing them.

5.4 Data model

. . . principal relations

- `users(id, email_hash, roll_no, dept, batch, role, created_at)` with a unique index on institutional email.
- `gigs(id, poster_id, category, title, scope, budget_paise, deadline, state, created_at)`, indexed on `(state, category, created_at DESC)` for the browse query.
- `bids(id, gig_id, worker_id, amount_paise, eta_hours, state)` with `UNIQUE(gig_id, worker_id)` so a worker bids at most once per gig.
- `contracts(id, gig_id, bid_id, amount_paise, state, confirm_deadline, revisions_used)` carrying the frozen terms.
- `accounts(id, owner_id, kind)` and `ledger_entries(id, txn_id, account_id, amount_paise, created_at)`, append-only.
- `status_history(id, contract_id, from_state, to_state, actor_id, created_at)`, the evidence spine and the source of every metric.
- `deliverables(id, contract_id, object_key, sha256, bytes, uploaded_at)`; the hash makes silent substitution of a file after a dispute is raised detectable.
- `disputes(id, contract_id, raised_by, reason, outcome, resolved_by, resolved_at)`.
- `ratings(id, contract_id, rater_id, ratee_id, score)` with `UNIQUE(contract_id, rater_id)`.
- `idempotency_keys(key, user_id, endpoint, response_body, created_at)`.

5.5 API surface

. . . REST design

Authorization is enforced per route by role *and* by relationship: only the contract's poster may confirm, only its worker may deliver, and administrators are the only role able to reach resolution routes. File access uses short-lived presigned URLs issued only to contract participants, so object storage keys are never guessable entry points.

Method	Path	Notes
POST	`/auth/register`, `/auth/verify`	institutional email + OTP
GET	`/gigs?category=&state=`	cursor paginated browse feed
POST	`/gigs`	poster only
POST	`/gigs/{id}/bids`	worker only, one per gig
POST	`/gigs/{id}/accept`	**money-affecting**, idempotent
POST	`/contracts/{id}/deliverables`	presigned upload, then commit
POST	`/contracts/{id}/confirm`	**money-affecting**, idempotent
POST	`/contracts/{id}/revision`	bounded by `revisions_used`
POST	`/contracts/{id}/dispute`	either party
POST	`/admin/disputes/{id}/resolve`	**money-affecting**, admin only
GET	`/me/ledger`	derived balances + entry history

Table 2: Principal endpoints. The four money-affecting routes are the only ones that write ledger entries.

5.6 Trust and verification *... non-research*

- Identity is established through institutional email verification, binding one account to one enrolled student.
- Reputation is computed from settled contracts only, using transparent counts (completed, disputed, cancelled, on-time rate) rather than an opaque score.
- Collusive rating inflation is limited by rule: ratings between the same pair of users count with diminishing weight after the first contract, and raw counts are displayed alongside the aggregate.
- New accounts operate under a bounded exposure cap (maximum concurrent active contracts and maximum single-contract value) until a minimum completion history exists.

5.7 Dispute resolution *... evidence-driven*

Disputes are resolved by an administrator, not by an algorithm. The engineering contribution is the evidence trail: every message, deliverable upload with its content hash, deadline, and state transition is timestamped and immutable once written, so the resolver sees an ordered factual record. Resolution outcomes are constrained to the three terminal edges of Table 1, each of which is a balanced ledger operation.

5.8 Web architecture *... web-based solution*

A typical 3-tier web architecture with clear role separation:

- **Frontend:** responsive SPA covering the poster flow, the worker flow, and the administrative dispute console as distinct routed surfaces with role-gated navigation.
- **Backend API:** stateless REST service holding all business rules; the client is never trusted to compute an amount, a state transition, or an authorization decision.
- **Database:** single relational instance with the constraints and indexes described above.
- **Object storage:** deliverables kept outside the database, referenced by key and content hash.
- **Scheduled worker:** a single background process for confirmation-window sweeps and nightly ledger reconciliation.

5.9 Testing and correctness assurance *... how we prove it*

- **Transition tests:** every cell of Table 1 has a passing test, and the complement (all state–trigger pairs *not* in the table) has a rejection test, so illegal transitions fail loudly.
- **Property-based tests:** randomly generated sequences of API calls are replayed against a test database, asserting after each step that all `txn_id` groups sum to zero and that held balances reconcile against non-terminal contracts.
- **Concurrency tests:** deliberate races (simultaneous confirm and timeout release, double-tap accept) run against a real database instance, asserting exactly one settlement occurs.
- **Reconciliation job:** a scheduled check recomputes balances from the ledger and compares them against the projection, alerting on any divergence.

6 Evaluation Criterion *... measurable and attributable*

We define success using measurable metrics tied to the core workflow.

6.1 Primary evaluation metric

Settled Completion Rate (SCR): the proportion of accepted contracts that reach `RELEASED` or a mutually agreed `REFUNDED` terminal state without administrator intervention.

- Target: SCR $\geq 85\%$ of accepted contracts during the pilot window.
- Attribution: computed from terminal states and timestamps in `status_history`, independent of any participant's self-report.

6.2 Correctness requirement *... non-negotiable*

Ledger integrity: 100% of ledger snapshots must balance to zero, and held funds must reconcile exactly against contracts in holding states. This is a correctness invariant rather than a target metric; any violation is a defect, and it is asserted both in the test suite and by the nightly reconciliation job.

6.3 Secondary evaluation metrics

- **Time-to-first-bid:** median time from gig posting to first bid, measuring whether discovery genuinely outperforms group chats.
- **Dispute rate and resolution time:** proportion of contracts entering `DISPUTED`, and median time from dispute raised to resolution.
- **Auto-release incidence:** proportion of contracts settled by confirmation-window timeout rather than active confirmation, indicating poster disengagement.
- **Perceived safety:** a simple 1–5 feedback question to both parties on whether they felt protected against non-payment or non-delivery.
- **Reliability:** availability of the API during the pilot (e.g., $\geq 99\%$ uptime in business hours), with zero tolerance for availability loss while funds are held.

6.4 Pilot validation plan *. . . high-level*

- Recruit 20–30 pilot users spanning both roles, drawn from at least two departments so that demand categories differ.
- Run the pilot on a closed-loop credit ledger with a fixed seeded balance per user, so the full escrow lifecycle is exercised without real fund custody.
- Deliberately seed 3–5 dispute scenarios (late delivery, partial delivery, scope disagreement) to exercise the resolution path and measure resolution time against known ground truth.
- Collect baseline expectations via short pre-pilot interviews on how such work is currently arranged and how often it goes wrong.

7 Scalability *. . . with foresight*

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in users and concurrent transactions by:
   - decoupling components (frontend/backend) behind a stateless API, so instances can be added horizontally without session affinity,
   - cursor-based pagination on the browse feed rather than `OFFSET`, whose cost grows with page depth,
   - keeping the ledger append-only so writes never contend on a mutable balance row; contention is confined to the single contract row being settled,
   - covering indexes on `(state, category, created_at DESC)` so the hot query is index-only,
   - isolating file traffic from the database by having clients upload directly to object storage via presigned URLs.
2. **Operationally deployable (practically):** The system should run on common web hosting:
   - managed relational database with transactional guarantees adequate for ledger operations,
   - containerized deployment with versioned schema migrations applied as an explicit, reversible step,
   - a staging environment carrying seeded data so that migrations and settlement paths are rehearsed before production.

   Onboarding a second campus should require configuration of the email verification domain, not code changes.

8 Engine availability heuristic

A mature set of "engines" (tooling/services) is available to implement this as a web-based project, so no new infrastructure needs to be invented.

Layer	Choice	Reason for the choice
Frontend	React with Vite, TypeScript, Tailwind CSS	Component reuse across three role-specific surfaces; static typing catches state/role mismatches at compile time.
Backend	FastAPI (Python 3.11), Pydantic	Declarative request validation and generated OpenAPI docs; typed schemas keep money as integers end to end.
Database	PostgreSQL 15	The only component that genuinely must be chosen carefully: row-level locking, deferrable constraint triggers, and partial unique indexes are all load-bearing for the ledger.
Persistence layer	SQLAlchemy 2.0 with Alembic	Explicit transaction boundaries and locking hints; migrations are versioned and reversible.
Auth	JWT access/refresh, Argon2id hashing, email OTP	Stateless verification suits horizontal scaling; OTP over institutional mail is the campus trust primitive.
Object storage	S3-compatible (MinIO in dev), presigned URLs	Keeps large files off the database and off the API path.
Background jobs	APScheduler (single worker)	Confirmation sweeps and reconciliation are low-frequency; a queue broker would be unjustified complexity.
Testing	pytest, Hypothesis, Testcontainers	Property-based sequences for ledger invariants; a real Postgres in tests, since locking behaviour cannot be tested against a stub.

Table 3: Technology stack and the reason each component was selected.

8.1 Selected technology stack

8.2 Justification

Only the database choice is technically constrained; the remaining components are interchangeable with equivalents the team already knows. PostgreSQL is retained specifically because `SELECT ... FOR UPDATE`, deferrable constraint triggers, and partial unique indexes let correctness live in the schema rather than depending on application discipline alone. Using these mature components lets us concentrate on lifecycle correctness, ledger integrity, dispute evidence, and measurement.

9 Project scope and deliverables *... iteration-friendly*

9.1 Initial deliverable *... first iteration*

- Institutional email verification and profile creation
- Gig posting, browsing with filters, bidding, and single-bid acceptance
- Contract state machine with hold and release, backed by the append-only ledger
- Deliverable upload via presigned URL and poster confirmation
- Timestamped `status_history` for SCR and time-to-first-bid measurement
- Transition and property-based test suites, including the ledger balance invariant

9.2 Subsequent deliverables

- Dispute flow and administrative resolution console with evidence timeline

- Reputation surface with completion counts, on-time rate, and collusion-dampened ratings
- Revision requests with a bounded revision count and deadline extension rules
- Notification layer (email optional in a later iteration; can start with in-app notifications)
- Category-level analytics: demand by category, median clearing price, unfilled gig rate
- Designed-but-deferred payment gateway adapter behind the existing ledger interface

10 Risks and mitigations

- **Fund custody / regulatory exposure:** holding real money on behalf of users is a regulated activity. Mitigation: the pilot operates a closed-loop credit ledger with no real custody; the gateway integration is designed as a replaceable adapter and deliberately left unimplemented, so the engineering artifact stands without the legal exposure.
- **Academic integrity:** a campus work marketplace can drift into contract cheating if gigs such as "complete my assignment" are permitted. Mitigation: an explicit prohibited category list enforced at posting time, mandatory acknowledgement, a reporting mechanism on every gig, and administrator takedown authority. This is treated as a product requirement, not a disclaimer.
- **Double-spend and race conditions:** concurrent confirm, timeout, and dispute requests could corrupt held balances. Mitigation: row-level locking with post-lock state re-read, single-transaction state-plus-ledger writes, idempotency keys, and explicit concurrency tests.
- **Collusion and fake reputation:** paired accounts could transact repeatedly to inflate ratings. Mitigation: diminishing weight on repeat-pair ratings, raw counts displayed alongside aggregates, and exposure caps on new accounts.
- **Off-platform leakage:** both parties have an incentive to settle outside the platform to avoid fees. Mitigation: run the pilot at zero fee, so measured leakage reflects genuine workflow gaps rather than price avoidance.
- **Cold start / thin liquidity:** a marketplace with no supply is not testable. Mitigation: seed both sides during recruitment and restrict the initial launch to two or three high-demand categories.
- **Evaluation measurement ambiguity:** terminal states must be fixed before development so SCR is comparable. Mitigation: freeze Table 1 in the first iteration and enforce it server-side.
- **Schema migration during a live pilot:** a migration applied while funds are held could strand a contract. Mitigation: rehearse every migration against seeded staging data, keep migrations reversible, and apply them only when no contract sits in a holding state.

11 Summary

This project proposes a deployable web platform that converts informal campus freelancing into a verified, escrow-backed, auditable transaction workflow. It meets the course criteria by emphasizing time-to-value through rapid iterations and by using measurable evaluation criteria, especially settled completion rate, with ledger integrity as a hard correctness invariant. Its engineering substance lies in an explicitly enumerated contract state machine, an append-only double-entry ledger in integer minor units, layered concurrency control against double settlement, and evidence-driven dispute resolution, rather than in research-driven novelty.